

Network on Chip Algorithms for Accelerating Block Sparse Linear Algebra on Tenstorrent Hardware

Rahmy Salman
North Carolina State University
Raleigh, USA
rsalman@ncsu.edu

Jiajia Li
North Carolina State University
Raleigh, USA
jiajia.li@ncsu.edu

Abstract—jli:[Add Frank, Ray, Joanthan (TBD), and Murali (TBD) names on this paper.]

jli:[We emphasize on SpMM for the majority of the paper, then talk about SDDMM as an extension.]

jli:[Real-data. For Jiajia.]

I. INTRODUCTION (1.5 PAGE)

jli:[Don't put much time in intro now, but feel free to write down your thoughts if any.]

With very little tooling and just the basic capabilities of the programming model, we write block sparse matrix multiplication kernels on Tenstorrent's network-on-chip architecture which outperform Nvidia cuSPARSE kernels on low density regimes when comparing against processing element counts with equal hardware acceleration throughput of matrix multiplication. Our data movement framework is designed to be lightweight on host preprocessing while achieving up to 89% the throughput of Tenstorrent's hand-tuned dense GEMM kernels. This result demonstrates the ability of the dual-NoC data movement system coupled with general-purpose processing units to orchestrate data movement on the fly and achieve near maximum compute saturation on irregular input data.

A. Motivation

Sparse linear algebra primitives are essential in numerous scientific applications.

The Tenstorrent Wormhole (and Blackhole) cards represent a leap in consumer-accessible AI acceleration hardware.

B. Key insights and contributions

In this work, we present a rigorous methodology for designing, implementing, and profiling algorithms for Tenstorrent chips and similar network-on-chip architectures.

- **Design, Analysis, and Implementation of Communication Primitives for Irregular, Blocked Computation**
- **Profiling Methodology for Identifying Bottlenecks in Full-Chip Programs**
- **Algorithms for Block Sparse Linear Algebra**
- **Performance Comparison to Nvidia Tensor Core Units**

C. Related Work

Current explorations of scientific algorithms on the Tenstorrent chip fail to utilize the most powerful aspects of the chip in two ways. First, they target the SIMD Vector Engine as their primary compute unit instead of the much more powerful matrix acceleration unit, the Matrix Engine. Second, they fail to explore data movement schemes for their applications, instead either investigating only single-core performance or settling on naive data movement across the chip

D. Experimental Methodology

We conduct robust profiling sweeps of per-core and per-chip runtime of our implementations on the Tenstorrent Wormhole device.

E. Limitations of the Proposed Approach

Unique challenges associated with the Tenstorrent architecture preclude the introduction of a robust performance model. These include per core clocks, nondeterministic fine-grained packet routing, and network asymmetry.

II. BACKGROUND (1.5 PAGE)

A. Block Sparse Linear Algebra

Why block sparsity: popular in applications, also hardware friendly (including multiple GPUs, AI accelerators). Important in quantum many-body problems.

Introduce SpMM and a bit about other kernel, such as SDDMM, SpGEMM.

... What level of background do we assume the reader has? BLAS (Basic Linear Algebra Subprograms) is a suite of software routines for performing basic vector and matrix operations. BLAS categ

B. Tenstorrent Network on Chip Architecture

The Wormhole card is the second generation of network-on-chip architectures created by Tenstorrent. The chip consists of 64 processing elements called "Tensix tiles" connected in a bidirectional network arranged in a 2D torus. Tensix tiles are composed of a highly performant matrix engine (FPU), a vector engine (special function processing unit, SFPU), and 5 independent "Baby RISCv cores." The matrix and vector engines are often named together as the "compute engine." The Baby RISCv cores are designed with area and

energy efficiency in mind, and their role in computation is to coordinate calls to the matrix and vector engines' acceleration capabilities. Each Tensix tile contains 1.5MB of local and private SRAM which it manages as a buffer between global chip DRAM and local registers. This memory hierarchy is cacheless: a Tensix tile uses two of its Baby RISC-V cores to explicitly perform data movement between global DRAM and private SRAM, and from its own SRAM to other Tensix tiles' SRAM banks in the grid of Tensix tiles (called the "core grid"). Each of the 5 Baby RISC-V cores has its own instruction and data memory for local variables it uses to manage calls to data movement and compute engine APIs. TT-Metalium is Tenstorrent's software stack for kernel development and comprises host-side and device-side APIs.

1) *Intra-tile Data Movement*: Each Tensix tile is its own independent processing element equipped with routers to both NoCs, a compute engine for linear algebra and special functions, a large SRAM bank, and 5 Baby RISC-V cores to run device kernels to manage these components. Baby RISC-V cores are in-order, scalar, single-threaded compute units: multithreading on a Tensix tile is a direct mapping of one thread to one Baby RISC-V core. These 5 Baby RISC-V cores have access to different hardware components and are the target of different device kernels. The BRISC and NCRISC cores each have access to SRAM and one of the two routers to one of the two NoCs. Most TT-Metalium programs assign NoC 0 to one of these cores and NoC 1 to the other, with the exact matching typically being empirically determined and dynamically set for a given problem and dataset. The remaining three cores (TRISCs 0, 1, and 2) do not have access to either router and instead manage the interactions between SRAM and the two workhorses of the compute engine: the matrix and vector engines. The three TRISC cores form a pipeline of UNPACK, MATH, and PACK operations wherein data is first "unpacked" from SRAM to input registers, then "math" operations are performed on the data using the matrix or vector engine, and finally the results are "packed" to SRAM from destination registers. Kernel developers need not explicitly manage these three cores. Instead, a single compute kernel comprised of C++ code augmented with proper API calls to compute operations gets JIT compiled to the three TRISC cores. In the case that the kernel writer requires explicit control over each of the three TRISC cores, there exist C++ macros to specify that a given region of code be executed on a specific TRISC core.

Program flow on a Tensix tile is synchronized around user-defined SRAM data structures called Circular Buffers (CBs). A TT-Metal kernel will define a set of CBs to exist on a set of Tensix tiles' SRAM regions at JIT compile-time. At runtime, data movement kernels compiled to BRISC and NCRISC cores of the Tensix grid will make direct memory access read requests to fetch data from a chosen location in the NoC and store that data in their tile's private CBs. The TRISC kernels will be waiting on those CBs for data to be present and perform their computation only when the data arrives. In most TT-Metal programs, input CBs are double buffered, thus

allowing for overlap between computation on the first set of input and data movement of the next. In practice, the only limit on overlap between data movement and computation is the size of the SRAM region on each Tensix tile.

2) *Inter-tile Data Movement*: The 64 Tensix tiles are arranged in a 2D torus connected by two independent networks, NoC 0 and NoC 1. NoC 0 is comprised of outgoing connections to the east and south directions, while NoC 1 is comprised of outgoing connections to the west and north. Both NoCs can be used to pass data between Tensix tiles, read data from DRAM to Tensix tiles, and write data from Tensix tiles to DRAM. This allows for overlap between reading and writing data to and from DRAM as well as between reading different data from any source in the NoC (other Tensix tiles, DRAM, other chips).

In addition to CBs forming the basis of intra-tile thread synchronization, the combination of CBs and semaphores in SRAM forms the basis of inter-tile communication. Dynamic addressing APIs allow tiles to read and set semaphores which live in other tiles' SRAM, allowing for fine-grained synchronization between Tensix tiles. When workloads are compute-bound, this synchronization overhead is hidden by the overlap between each tile's compute and data movement cores **[TODO: verify this claim... it certainly seems perfectly true to me...]**.

3) *Networked Data Movement*: Where is DRAM

How to read from DRAM

Where is SRAM

How to read from SRAM

Where is other cores

How to read from other cores

III. ALGORITHM DESIGN (3 PAGE)

A. Overview

add a workflow figure.

[TODO: Wait so actually SnF and CDA and whatever we're doing for SDDMM are all the same operation with different levels of synchronization. SnF requires the most neighbor-to-neighbor sync and zero barrier CDA with variable reduction dimensions requires the most frequent barriers SDDMM CDA requires asymptotically fewer barriers than SpMM CDA.]

~~Irregular data access patterns necessitate dynamic movement. We design our data movement framework around two guiding principles: host preprocessing must be kept small and DRAM accesses ought to be minimized. We interpret the indices of the block sparse matrix as compact, jagged matrix of indices. Indices in the same row of this jagged matrix correspond to work which will be assigned to Tensix tiles in the same row of the Tensix grid, and likewise for the columns. Rows of this virtual, jagged matrix correspond truly to rows of the sparse matrix, but column the i^{th} column of the virtual, jagged matrix represents the i^{th} nonzero entry of each row of the matrix. This virtual, jagged matrix is the natural result of iterating over the data vector of the sparse matrix, making it computationally trivial to obtain. We use only this~~

representation of the sparsity pattern of the matrix to expose data-sharing opportunities on the device.

B. Store and Forward

jli:[Does this technique being released by Tenstorrent?]

jli:[Need more details for this para. Could you put a draft figure here?] Store and forwarding is a technique for sharing a block of data across a statically-determined set jli:[any particular meaning for statically-determined?] of Tensix tiles. TT-Metal provides a software solution for broadcasting jli:[State more for broadcasting. We still have the broadcasting code right? If so, could we include broadcasting as a baseline?] data across rectangles of the Tensix grid, but this function incurs unnecessary jli:[What's that?] synchronization overhead for our use case. Within our block sparse kernels, we use store and forwarding for situations where individual rows of the Tensix grid would all request the same block of data jli:[What data? A,B,C?] from DRAM. At [TODO: define] device kernel compile time, one column of Tensix tiles is designated as the "injectors" which perform NoC requests to DRAM and forward the obtained data across their respective rows of the Tensix grid. The choice of column is affected by the NoC selected for the store and forward data movement: the leftmost column is used for injecting when using NoC 0, and the rightmost when using NoC 1. The remaining cores are "receiver" tiles which obtain data from a neighboring tile instead of from DRAM, thus eliminating redundant DRAM reads. Once a receiver tile obtains its data, it stores that data in its private SRAM region and forwards that same data the next Tensix tile in its Tensix tile row. The final Tensix tile is the "sink" and does not forward any data. This pattern is used in SpMM for the data movement kernel charged with reading blocks of the sparse matrix into the Tensix grid.

C. Dynamic Direct Addressing

Store and forward has limited applications when data sharing opportunities are irregular and variable across a single run of a device program, so more fine-grained techniques must be employed for reducing DRAM accesses and NoC congestion in most sparse algorithms. To this end, we augment the store-and-forward method of broadcasting data across a set of compute units to support irregular multicasting across concurrent "share sets" of Tensix tiles discovered by Baby RISC-V cores at runtime. We call this method Dynamic Direct Addressing and identify two variants for use in our block sparse algorithms.

Store-and-forward minimizes both DRAM reads and NoC contention when data sharing is regular and easy to discover at compile time. These attributes make store-and-forward an excellent match for data movement of the sparse matrix in SpMM, but this method fails to generalize when not all Tensix tiles in a given row or column seek to read the same block of data from DRAM. This is the case when reading the dense matrix in SpMM as well as all of the data movement for SDDMM. jli:[Explain more for the above words.] For these cases, we implement Dynamic Direct Addressing (DDA). In

DDA, each Tensix tile reads the indexing data that its potential share set members will use to make read requests at each iteration and determines its share set, role, and sender and receiver tile in the share set dynamically for each new data sharing opportunity. Before the collective computation of each share set, each row (or equivalently, column) of the Tensix grid must commit to an all-to-one synchronization barrier to avoid races to semaphores. This barrier is necessary in DDA but not store-and-forward because the sender/receiver relationships between Tensix tiles in DDA vary at each new data sharing opportunity. Additionally, TT-Metalium puts a limit of eight on the number of semaphores a single Tensix tile can locally declare. If this limit were increased to $2(m + n + 1)$, where m and n are the number of Tensix tiles in a row and column of the Tensix grid respectively, then we could let each Tensix define a sender and receiver semaphore for each of the cores it may potentially share data with, enabling DDA to run without any barrier. jli:[Put this potential at the end of this subsection.] In SpMM and SDDMM, each nonzero block of the sparse matrix presents a new data sharing opportunity along the dense input matrix(ces). Since SpMM reads only one block of the dense matrix for each nonzero block of the sparse matrix, a barrier must be performed after each set of input blocks is read for a step in the multiplication. Likewise, since SDDMM reads an entire row of the dense matrix C and column of the dense matrix D for each nonzero block of the sampling mask, the barrier only needs to be performed after an entire set of output blocks is computed. We find that this barrier is a worthwhile tradeoff in SpMM, with only a few degenerate test cases exhibiting slight loss of performance when compared to naive data movement on the dense matrix.

We can quantify the efficacy of a data movement scheme by the number of DRAM requests it makes and the number of hops on the Tensix grid it makes starting from the injector cores which make the DRAM requests. Store and forward is optimal in both regards: for each nonzero block of the sparse matrix, it makes exactly one DRAM request, and it make one hop for each Tensix tile using that input data. There are no redundant DRAM requests and there are no extraneous NoC hops taken. On the other hand, we can consider a naive data movement scheme in which no data is shared and each Tensix tile makes its own NoC request to DRAM for each input block it processes to be the worst case data movement scheme. With the notable exception of block-banded matrices, CDA's efficacy tends to approach that of store and forward's as matrix density increases. In the worst case where CDA exposes no data sharing, we incur the same number of DRAM reads and NoC hops as the naive data movement scheme while incurring the overhead of calculating the lack of share sets during runtime. However, with even minimal amounts of data sharing exposed by CDA, performance improves significantly over naive data movement [TODO: this is what the fill-row experiments do, now just quantify them.].

This pattern is used for reading input blocks of the dense matrix in SpMM. [TODO: do we have to talk about the synchronization step in CDA and so on?]

A. SpMM

SpMM is a foundational kernel in many scientific applications. We implement an optimized SpMM algorithm where the sparse matrix is in Block Compressed Sparse Row format, allowing us to utilize the 1.024 TFLOPS of BF16 throughput for each Tensix tile's matrix engine. In synthetic data dense enough to saturate the Tensix grid, we achieve throughput within 87% of our measured GEMM benchmarks.

In SpMM with single-element sparse indexing formats like CSR and COO, there are two large challenges for achieving peak performance: irregular memory access and low computational intensity. Block sparse indexing inherits these problems, but at densities as low as 20% the computational intensity of multiplying many nonzero blocks together is great enough for a pipeline of data movement to form across the Tensix grid, leading to performance which approaches the chip's theoretical peak.

[TODO: find your favorite figures and nerd out]

[TODO: the two paragraphs above expose the need for a performance model. How can I speak of saturating the core grid without either measuring that saturation (how? can we quantify pipelining? Yes, we can count how often we wait on CBs and push to the back of non-empty CBs...) or arguing for its existence through a theoretical framework (ahem, a performance model) which predicts performance based on the attributes of the data and routing scheme actively running on the chip. At any rate keep speaking!]

The TT-Metal software suite integrates the open-source Tracy profiler for minimally invasive performance measurement on the host and device. We utilize Tracy for low-overhead runtime measurement and for benchmarking functionality of individual hardware components on the device.

V. PERFORMANCE PROFILING METHODOLOGY

Tenstorrent provides a build of the open source profiler Tracy for capturing per-core performance data of the chip.

A. Matrix Engine Throughput

We contextualize the performance of Tenstorrent's family of cards by measuring throughput of their matmul acceleration units and compare them to widely used Nvidia Tensor Cores.

B. Kernel Microbenchmarking

To systematically identify bottlenecks in the runtime of device code, we employ a microbenchmarking methodology wherein individual Baby RISC-V cores omit the main operation of their kernels while maintaining responsibilities to measure what the runtime of the device code would be if each kernel were perfectly optimized.

As part of the development and optimization workflow, we describe a simple benchmarking technique which is highly effective for identifying bottlenecks in target algorithms, leading to well-informed designs for new optimizations. Independent clocks for each Baby RISC-V core, hidden dead tiles in the

Tensix grid, and other natural sources of variation in the computational throughput and DRAM bandwidth of Tensix tiles make it nearly impossible to reason about performance of algorithms on a cycle-to-cycle basis, but this does not limit the design space of optimizations on the architecture. Instead of counting cycles, programs are to be analyzed by the time spent at synchronization points both within and between Tensix tiles.

The profiling plan utilizes two programming abilities. First, the program must be able to measure the execution time of a device program. This can be obtained using standard C++ timing libraries, or through Tenstorrent's attached build of the open source profiler Tracy. Second, the program must be able to selectively omit arbitrary portions of the algorithmic pipeline on a single Tensix core. This can be achieved by rewriting device programs which run only the device code which handles CB synchronization. By using the JIT compilation feature of TT-Metal programs, we can use C++ macros to define which regions of the code can be omitted in any particular run in the microbenchmarking suite without having to write and store separate kernel files for each such run. This technique is universal because all nontrivial TT-Metal programs follow this CB-oriented synchronization structure. Since the architecture is designed to allow for overlap between the three RISC-V units on each Tensix tile, this method does not measure the time each RISC-V unit is active throughout the duration of the algorithm. Instead, this method provides a lower bound on the total runtime of the algorithm given an implementation of the chosen computation which runs at infinite speed. Attempted methods to measure the idle time of each of the RISC-V cores were limited due to the Tracy profiler's limit on zone instances for a single RISC-V core. The primary information gathered by this method is the **lonely worker time** **[TODO: define "lonely worker time". It's the opposite of computational overlap between two kernels.]** of each main operation on a RISC-V core. This technique finds the **lonely worker time** aggregated across the core grid, so any reasoning about runtime behavior of individual cores must be held orthogonally to this method.

The main operation for the compute kernel is performing the matrix multiplication between the two input blocks and accumulating the results in an intermediate SRAM buffer. For the two data movement kernels, the main operation is reading a single input block from the NoC, regardless of location, be it DRAM, a neighboring Tensix tile, or an arbitrarily chosen Tensix tile. For one DM kernel, there is a second main operation: writing back output blocks to DRAM. We found that performance was maximized by assigning this second main operation to whichever DM kernel was handling the reads of the sparse matrix data.

[TODO: Figure X: illustrate the logical shutting off of a RISC-V core and how that affects runtime performance (see green notebook)]

Figure X shows an example of **lonely worker time** on the compute kernel in a test case with equal sizing between blocks of the sparse matrix and blocks of the dense matrix and where the size of the input blocks is large enough to put

significant load on the compute kernel. This figure assumes a data movement scheme which is balanced, incurring the same latency to read a block from the sparse matrix (row 0) as to read a block from the dense matrix (row 1). This set up (equal block sizing with identical data movement schemes) corresponds to two of our tested algorithms **[TODO: name those two algorithms here]**. Numbered regions in the figure correspond to instances of the main operation of each kernel and are the target of omission in this microbenchmarking technique. In our benchmarking suite, we take turns omitting the main operation of each of the three kernels. In the case of Figure X, omitting either DM kernels' reads will not save anytime because the two read operations constitute a perfectly overlapped set of operations, and the compute kernel is bottlenecked by the slowest of the two reads. By omitting the compute kernel's main operation, we observe a reduction in the execution time of this test case which gives us a hint to the amount of time the core grid spends in aggregate with both data movement kernels idle.

[TODO: word this thought more professionally] What's really being measured by this technique is the difference in execution time of the algorithm in aggregate with the same operation omitted from each of the Tensix cores and the full algorithm. Figure X shows the time each Baby RISCv core might be active during the execution of a single Tensix core. The difference in execution time between the full algorithm and the microbenchmark is determined by the slowest Tensix core in the two cases. There will always be Tensix cores which complete their computation sooner than others due to asymmetry in DRAM and NoC requests, so any investigation into the performance across the core grid should include discussion on the placement and role of each Tensix core in the data movement scheme of the algorithm.

In such a case as Figure X, we can conclude that to reduce the execution time of the algorithm on this particular test case, an algorithm designer would have to optimize the operation of the compute kernel. In practice, we did not observe any block sizing which resulted in a balanced data movement cost between the two input matrices when each was assigned its own NoC. This is due to imbalance in the two on chip networks which has been documented in experimental kernels by the TT-Metalium team.

VI. EXPERIMENTS (2.5-3 PAGES)

We evaluate the performance of the Wormhole card on our suite of block sparse algorithms and compare to the Nvidia RTX 4090 GPU.

A. Platforms and Dataset

Hardware Configuration/Specs, including GPUs. Software environment. Time evaluation methods: timing (cite the repo here), report the evaluate over XX runs.

Dataset: synthetic and real.

B. Overview Evaluation

Best Timing and GFLOPs, vs. machine peak and GPU, bandwidth numbers. Explain why we are better, closer to peak; and the opposite. figure here.

C. Analysis

1) *Three approaches comparison:* Versions: baseline (directly from DRAM), SNF (sparse A using it), SNF+DDA (for both input matrices), SNF+DDA+LB. More detailed analysis figure here

If profiling or modeling.

2) *Preprocessing Overhead:* Host side overhead (data transfer), preprocessing (sorting) overhead, argument parsing overhead.

Device side overhead:

3) *Scaling [Lowest priority]:*

4) *Profiling Results:*

D. Applicability: SDDMM

jli:[Only need to introduce SDDMM as a standalone section in later part of the paper.] Sampled Dense-Dense Matrix Multiplication (SDDMM) is the workhorse computation in algorithms for graph analytics and factor analysis. SDDMM takes a masked product of two dense matrices where the mask has arbitrary shape. The sparse mask forms a working set which bears some similarity to SpMM but differs in the following ways. First, the work required to compute each nonzero output block is uniform. Second, include a figure.

VII. CONCLUSION AND FUTURE WORK

Mention block-sparse tensor work in the future.

RWS

REFERENCES

- [1] H. Kopka and P. W. Daly, *A Guide to L^AT_EX*, 3rd ed. Harlow, England: Addison-Wesley, 1999.